

Care Note

Nightingale 72-Hour Build. Synthetic data only; not safe for real PHI as-is (§7).

1. What Care Note is, and how we approached it

A patient's story is currently scattered across dated free-text notes, each written by a different person in a different screen. Nobody holds the whole thread. A clinician opening a chart before a consult is reconstructing months of history by scrolling and guessing what matters.

Care Note replaces that with one shared, longitudinal record per patient. Clinician notes, nurse and staff notes, the patient's own contributions, and AI-scribed summaries of every consult all land in a single timeline, each entry carrying who wrote it, when, and what it came from. On top sits a **Glance View** — a top card designed to be read in under ten seconds: what changed since you last looked, what could hurt this patient, what matters and why, what is outstanding and whose it is.

Four commitments shaped every decision, and they pull against each other in ways worth being explicit about.

It has to be safe. These are medical records. Access control is fused to both role and clinic and enforced server-side, never in the UI. Every piece of text that reaches a model passes one redaction chokepoint that strips names, identifiers and phone numbers, re-scans its own output, and refuses to send rather than leak. Logs carry IDs and actions, never content.

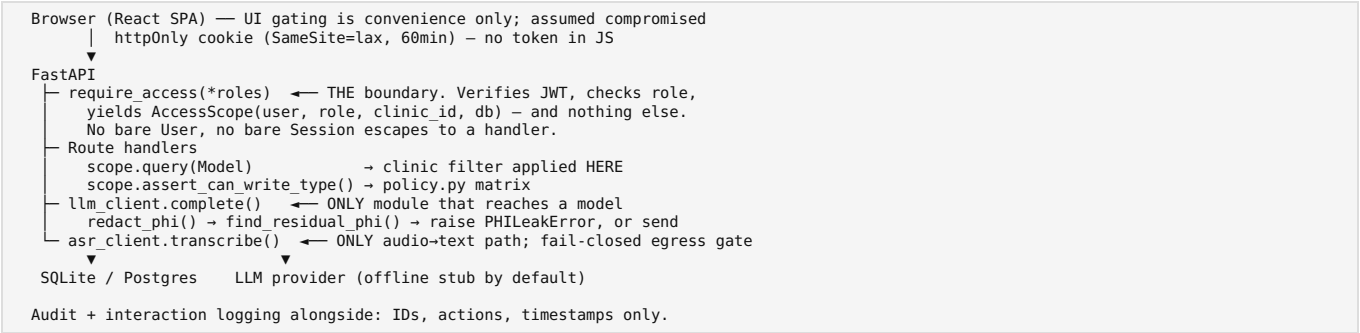
It has to be fast. A clinician sees a great many patients in a day, and a tool that costs thirty seconds per chart costs hours per week. The Glance View is measured, not asserted: P95 server handling of ~11 ms against a 300 ms budget (§4).

It has to be worth adopting. Nobody adopts software that makes their job harder, and clinical staff have every reason to be sceptical. So the system tries to *remove* work rather than add process: confirming or dismissing a suggestion is one click without leaving the card, the AI writes the consult summary nobody wanted to type, and role-based views mean each person sees their own job rather than everyone's. The patient view is written in plain language for an anxious reader, not clinical shorthand.

And the AI has to be continuously verifiable. This is the constraint that shapes the architecture, because in this domain neither direction of error is acceptable — a false positive trains staff to ignore the system, and a false negative is a missed allergy. We do not think that is solved by a better model. It is handled by treating **AI output as a claim, not a fact**: every claim has a source you can open, a confidence measured from evidence rather than self-reported, a human who can accept or reject it in one click, and no power to overwrite what a clinician wrote. Where the system cannot support a claim, it **abstains** — an untraceable summary line gets no citation rather than a plausible-looking one. That is a data-model commitment before a UI one, which is why it sits in the schema (§3) rather than in a component.

2. Architecture

A React client talking to a FastAPI backend over SQLite. Two things in the diagram do the real work: `require_access()`, the single door every request enters through, and `llm_client.complete()`, the single door every outbound piece of text leaves through. Everything else is ordinary. Both are chokepoints by design — a rule enforced in one place that nothing can route around is worth more than the same rule written correctly in forty handlers.



Python 3.11 / FastAPI / SQLAlchemy / SQLite (Postgres-ready), React 18 + Vite + Tailwind, pytest. FastAPI specifically because dependency injection is what makes the RBAC rule unforgettable:

RBAC fuses role and clinic inseparably. The common real failure is a route that checks role and forgets the tenant filter. Here that is not expressible. `require_access()` is the only way a route learns its caller — there is no exported `get_current_user`. It yields an `AccessScope`, never a User, and that is also the only DB handle a route gets. `scope.query()` applies the clinic predicate itself and **raises `TypeError`** on any model lacking `clinic_id` rather than returning it unfiltered. `clinic_id` comes from the verified JWT only. No handler in `patient_routes.py` mentions `clinic_id` in a filter. Cross-clinic fetches by exact id return **404, not 403** — a 403 confirms the id exists.

One redaction chokepoint. `redact_phi()` is called unconditionally by `complete()`, the only module that reaches a model. Callers cannot opt out; redaction is idempotent. After redacting, the payload is re-scanned and the call **raises rather than sends** if PHI survived. Three source-scanning tests fail the build if any other module imports an LLM SDK, references a model endpoint, or removes the redaction call.

3. Schema

Entry — one item on the patient's timeline — is the hub, and every relationship the requirements name hangs directly off it. Versions give revision history, Comments give collaboration, Highlights give the Glance View its content, and AIScribedNote is what marks an entry as machine-authored. Provenance is the one deliberate exception to normal database design: it is a string URI rather than a foreign key, because the things a citation points at are not all rows in one table. The learning mechanism reads from the same graph and is constrained by it, described at the end of this section.

Link	Mechanism
Entry → Versions	<code>Version.entry_id</code> , unique (<code>entry_id</code> , <code>version_number</code>). Full snapshots, not diffs — revert is a copy, and no chain can be corrupted by one bad link
Entry → Comments	<code>Comment.entry_id</code> ; threads via self-referential <code>parent_comment_id</code> ; open/resolved status
Entry → Highlights	<code>entry_id</code> + character span + the source_version_number the span was computed against , so an edit cannot silently move a highlight onto different text
Entry → AIScribedNote	One-to-one. This row's <i>presence</i> is what makes an entry AI-authored; <code>author_role='system'</code> is the denormalised fast check. A UI cannot render an AI note as a clinician's
AIScribedNote → transcript	Shared <code>session_id</code> with <code>TranscriptSegment</code>
Summary line → spoken words	<code>SummaryAttribution</code> links each summary line to the segment that produced it
Anything → Provenance	A string URI , not a foreign key

CLINIC <- USER, PATIENT, ENTRY, FEATURE_WEIGHT · PATIENT <- ENTRY, TASK, CAPTURE_SESSION · ENTRY <- VERSION, COMMENT, HIGHLIGHT, TASK, SUMMARY_ATTRIBUTION and -o AI_SCRIBED_NOTE, ENTRY_ARCHIVE · COMMENT <- COMMENT, TASK · USER <- INTERACTION_LOG, AUDIT_LOG, PATIENT_VIEW. Full Mermaid ER diagram in `SCHEMA.md`.

Provenance is a URI — `entry://<id>#span:<start>-<end>`, `session://<id>#turn:<n>`, `transcript://<id>#segment:<n>`. A foreign key points at one table; provenance targets are heterogeneous — a whole entry, a character range in one, a turn in an AI session, an audio segment that is not a row here. One resolvable string covers all and keeps "click a highlight, land on the source" to one code path. Cost: the DB won't enforce integrity, so `resolve()` is the only dereference path, it **raises on a dangling pointer** rather than degrading to empty, and it enforces `clinic_id` — a valid pointer must never become a cross-tenant read primitive.

clinic_id is denormalised onto every scoped table, even where a join would derive it. That redundancy is what lets `AccessScope.query()` apply one uniform predicate to any model — and why it can *refuse* one lacking the column.

Learning integration. `InteractionLog` records what clinicians touch as **extracted tags only, never prose**. `record_interaction()` calls `apply_signal()` in the same operation, so no route can log a signal the learning table never sees — the same chokepoint reasoning as redaction. Evidence aggregates per (`clinic_id`, `feature_tag`) into `FeatureWeight` with a 90-day half-life, saturating into (-1, 1), read by `scoring.learned_component()` as one term. `FeatureWeight` is a materialised view of the log, never nudged, so it cannot drift from its evidence. It is capped at 0.25 of the total, cannot invent a highlight (the rule layer runs first), cannot cross a clinic, cannot be trained by patients, and **cannot silence safety vocabulary** — allergy, sepsis, anaphylaxis and self-harm tags are floored at zero.

4. Glance View and latency

The Glance View is the product's core claim: that a clinician can be oriented in under ten seconds. Two things make that possible, and both are subtractive. The card **refuses to show most of what it knows** — a top card containing everything is just the timeline again — and expensive work happens on write rather than on read, so opening a chart is mostly a database read of precomputed rows. The latency figure below is evidence for that design choice rather than a boast about speed.

The Top Card answers four questions in fixed order: what changed since you were last here; what could hurt this patient; what matters and why; what is outstanding and whose it is. Ranking is a weighted sum over named features, each highlight showing its own arithmetic and a one-line risk_reason. **Nothing is ranked by a model.**

Measured: P95 ≤ 300 ms target. A middleware reports X-Response-Time-Ms per request — request in, queries run, payload serialised, response out. 200 iterations after 20 discarded warm-ups, 11-entry chart with 6 highlights, SQLite on local disk. Re-measured after the Phase 7 fixes touched every timestamp in this payload; three consecutive runs: **P95 11.54 / 11.09 / 11.63 ms**. Range reported, not the best run. These are lower than the 13.30–15.94 ms recorded before those fixes, which is container load rather than an improvement anyone engineered — attributing it to the change would be reading noise as a result.

This **excludes network transit and browser render** — those depend on deployment and device, and folding loopback in would invent precision. The client measures its own round trip and the header shows both, so the demo never conflates them.

The figure is evidence for something narrower than “fast”: **application work is a small fraction of budget** and no N+1 hides in the hot path. Two decisions carry that — **highlight scores are computed on write, not read**, so the view reads precomputed rows; and **timeline enrichment is batched** into four grouped queries regardless of chart size. Production means Postgres over a network, hundreds of entries and concurrent load; the ~20× headroom makes inversion unlikely, but the test that settles it is a loaded staging environment.

5. Trust calibration, evaluation and abstention

This is the section that answers “why should a clinician believe any of this?” Five mechanisms, each closing a different way an AI-assisted record can lose a user's trust: *unearned authority, invisible uncertainty, silently overwritten human work, contradictions nobody surfaces, and generated text reaching a patient. The table that follows asks the harder question of the three numbers we put on screen — a risk badge, a confidence label and an importance score — which is not what they are but how anyone would know if they were wrong, and what the system does when it cannot answer.*

1. Accept / reject. Highlight.status starts suggested and needs a clinician decision; no AI claim reaches the card as fact on its own authority. One click, inline, no navigation, immediate confirmation — because this decision is also the training signal, and a high-friction control starves the loop. The interaction cost is a design constraint, not a nicety. A clinician's own hand-marked span is recorded accepted on creation and carries a scoring bonus, so human judgement outranks machine suggestion on the card by construction — which is precisely what silently broke in Phase 7 (\$6).

2. Visible confidence, derived not asserted. Confidence is computed from hedging density in the source transcript on **both** paths — a live model's self-report is stored for calibration and is never what the clinician sees, because a model's opinion of its own reliability is not evidence about it. Bands are numeric and defined once: **high ≥0.75, medium 0.60–0.75, low <0.60**, and the chip shows the word and the number together. Low-confidence summaries **flag separately from risk** — “this might be dangerous” and “this might be wrong” are different warnings a clinician acts on differently.

3. Clinician precedence and a review flag. The brief allows either; we do both. A clinician edit wins immediately — care is never blocked on a resolution workflow — but conflict_flagged is set and supersedes_entry_id records what was overridden, so the disagreement stays visible. Precedence alone loses information: that the AI disagreed is *itself* clinically interesting, and discarding it quietly is how a system teaches users to stop trusting it. AI notes are never edited in place; corrections supersede.

4. Contradiction detection, including human-human. Mechanism 3 handles a disagreement that has been *resolved*. The dangerous one is unresolved: a nurse records a penicillin allergy, a clinician prescribes amoxicillin, neither is wrong on purpose, and no precedence rule applies because both are people. services/contradictions.py detects three classes deterministically — allergy against administration (including by drug class), dose disagreement on the same drug, and started-here/stopped-there — cites **both** entries, and **resolves nothing**. Deciding that the more recent note wins would silently discard an allergy recorded last year. It sits above everything else on the card.

5. Patient-facing generation is structurally impossible, not approved. Showing a clinician a hallucinated line is a bad day; showing a patient one is a different category of harm — no second reader, no provenance rail, no basis to doubt. Rather than generating patient text and requiring sign-off — a step people click through under load — no generated text can become patient-facing at all. PATIENT_FACING_TYPES is writable only by clinician; Role.SYSTEM can write nothing; assert_never_patient_facing() runs at import against the scribe's own type map so a future edit fails loudly; and AI types are absent from the patient's viewable set. A clinician may read an AI summary and write an instruction from it — they type the words.

Supporting all five: provenance_pointer is non-nullable on Highlight, resolution lands on the **character span** not the note, and AI-vs-human is carried by four independent signals (rail style, colour, typeface, label) so it survives in greyscale.

What each number means, and how we would know it was wrong

	Risk badge	Confidence label	Importance score
What it is	Ordinal none → critical. Deterministic rules over transcript text set a floor; a model may raise it, never lower it	A number in 0.35–0.90 measured from hedging density in the source, banded high/medium/low	Weighted sum of named terms (recency, risk, entities, open actions) plus a learned term capped at 0.25
How we'd know it was wrong	Same input must give same level every run; a floor test asserts chest_pain cannot resolve below high. Drift is visible because model_proposed_risk and risk_floor_applied are both stored	Confidence must fall as hedging rises — asserted against a plain and a hedged transcript. The band boundary and the UI's flag threshold are asserted to be the same constant	Every highlight shows its own arithmetic; a wrong ranking is inspectable term by term rather than argued about
What happens when it is	The rule floor holds the badge up. risk_floor_applied tells the clinician the level came from a rule, not a model's mood	Below 0.60 the summary is flagged “verify against source” separately from risk, and the provenance rail opens the transcript	The clinician accepts or rejects in one click; rejection dampens the tag — except safety vocabulary, floored at zero so fatigue cannot silence anaphylaxis
When it abstains	Unparseable model risk falls back to low, and the deterministic floor still applies on top	Never claims 1.0 — a summariser reading a transcript it did not hear has no business reporting certainty	A span with no clinical reason produces no highlight at all ; the rule layer runs before scoring

The same discipline governs the two places abstention matters most. **Attribution** classifies each summary line verbatim / derived / **nothing** — a line the model composed or invented gets no pointer and the UI says “no traceable source” rather than pointing somewhere plausible, because a false citation that looks checkable survives review. **Redaction** re-scans its own output and **raises rather than sends** if anything identifying survived; it is also tested for the opposite failure, that Metformin 500mg BD and HbA1c 8.2% survive intact, since over-redaction corrupts the note it was protecting.

Exposure bias is the one hazard we can only partly answer. Learning sees feedback only on spans it chose to surface, so a tag below the cut is never shown, never accepted, never weighted. One suggestion slot per entry is reserved for a candidate carrying a tag the clinic has never given feedback on — deterministic rather than epsilon-greedy, because a card that differs between loads is worse on a clinical surface than the bias it fixes. It narrows the loop; it does not close it.

6. Trade-offs, assumptions, deferred scope

Every decision below is one where a defensible alternative existed. They are recorded with reasoning rather than as a feature list, because in 72 hours the interesting information is what was traded away and why. DECISIONS.md holds the full log — around seventy entries, including scope deliberately cut and decisions later reversed. The section ends with the defects we found after calling the build finished, which is the part most worth reading.

Regex redaction over NER (D-012). Data is synthetic, so recall against real name diversity isn't what's tested — the boundary's un-bypassability is. Regex is auditable line by line, worth more in a trust system than F1 from an uninspectable model. Production layers NER behind the same signature.

Content is deliberately not HTML-escaped on write (D-015). Clinical prose contains BP <120/80 and dose <5mg. Escaping on write double-escapes on render; tag-stripping can eat <5mg and silently turn a dose limit into mg. Corrupting a note is worse than the XSS it prevents — because untrusted content is never rendered as HTML at all, enforced by a build-failing source scan.

Assumptions where the requirements were silent: staff cannot view clinician_sections (D-004, least privilege); admin reads all in-clinic but authors no clinical content (D-011), so it cannot quietly alter the record.

Optimistic locking over CRDTs. RBAC already partitions who writes what, so most conflicts are prevented by construction; presence is polish paid for in infrastructure.

Multilingual: partially closed, and the design point matters (D-058). Phase 5 carried code-switched speech through intact but tagged English only, so identical clinical content produced ['symptom:swelling'] in English and [] in Malay — no tags, no score, never on the Glance View. That fails in the worst direction: the patients least likely to be understood in English are the ones the system stops surfacing. A Malay vocabulary now maps each term to the **canonical English tag**, so bengkok emits symptom:swelling. That

is not cosmetic — tags are the keys Phase 4 learns against, and a separate symptom:bengkak would have split one concept into two features and stopped a clinic's learned attention transferring across the language its patients used. Still unbuilt: translation, non-English summary generation, every language but Malay. The fourteen terms need native-speaker and clinical review — the mechanism is proven, the word list is a demonstration. Testing it also surfaced that **negation is unhandled in both languages** ("Patient denies chest pain" tags chest pain); pre-existing, now pinned in both so a fix cannot be applied to one and not the other.

Deferred: multilingual summary generation and handwriting capture (D-019). Summary generation in a second language is a *time* deferral only. Handwriting OCR was deferred **structurally**: a different ingestion pipeline (image → OCR → redact → summarise) where redaction is materially harder on noisy output — one mis-recognised character defeats a pattern that would have caught an identifier — and medical handwriting OCR is hard even for well-resourced products. Ambient voice capture serves the same "fast unstructured capture" need more safely.

Defects found after the build was "done", and what they have in common. The final pass found every enum column is declared Mapped[StrEnum] but backed by String, so reloaded rows return plain str and three is comparisons were silently dead branches (D-055) — most visibly, superseded highlight suggestions were never deleted and the Top Card rendered every claim twice. Four more surfaced afterwards from someone simply using the thing (D-059–D-062): a clinician's hand-marked span vanished from the Glance View; confirming one suggestion made every other suggestion on the open card return 404; "new since your last visit" stayed empty for an entire first session and then stopped advancing for anyone who refreshed often; and a task could be raised and never closed. A fifth, found while reproducing those: every timestamp left the API without a UTC offset, so a browser read it as local time and a note written seconds ago rendered as "8h ago" in the timezone this was demoed in.

All of them survived a green suite, and for the same reason. Each lives in the seam between two pieces of individually correct code. The manual-highlight bonus is right where it is written and right where it is recomputed; only the ordering of the two is wrong. The timestamps are right in the database and right in the browser; only the contract between them was unstated. Component-level tests cannot see this class, which is why the regressions are written as end-to-end sequences — open the chart, write a note, reload — and why ten of the fifteen fail against the previous commit. A test that has never seen its regression is a description of current behaviour wearing a test's clothing.

Reported at this length because the honest version of "structural enforcement catches mistakes" has to include the classes it missed. The enum column-type migration and a real end-to-end browser test are both deferred rather than attempted hours before submission.

7. Security posture

One table, three honest statuses, no hedging. It is longer than a table of things that work would be, because roughly a third of the rows are gaps. That is deliberate: a control whose edges nobody knows is more dangerous than a weaker one everybody understands, and a reviewer who finds an undisclosed gap should be able to assume there are others. Everything marked "known gap" is something we would fix before this touched a real patient, and the summary below the table says plainly why it cannot yet.

Implemented = built, tested, verifiable here. **Documented decision** = deliberate for a 72-hour prototype, production shape stated. **Known gap** = genuinely missing, listed because a control whose edges nobody knows is worse than a weaker one everybody understands.

Area	Status
PHI redaction chokepoint	Implemented — regex + gazetteer, fail-closed
Stored-XSS / content safety	Implemented — never rendered as HTML, enforced by source scan
RBAC (role + clinic, server-side)	Implemented — fused, proven over HTTP
Logging hygiene	Implemented — content-free by construction, verified by grep
JWT storage	Implemented — httpOnly cookie, SameSite=lax, 60min TTL
AI note immutability; comment isolation from patients	Implemented — corrections supersede; comments refused at route <i>and</i> stamped internal
Conflict handling	Implemented — precedence <i>and</i> flag; disputed content never deleted
Audio never persisted; un-redacted egress	Implemented — memory only, asserted against the DB; egress gate fails closed, never degrades to stub
Learning substrate holds no prose; clinic-partitioned; safety floored	Implemented
Enum comparison correctness	Implemented (guarded) — == throughout; source scan fails the build (D-055)
CSRF defence	Documented decision — SameSite=lax only
TLS in transit	Documented decision — terminates at reverse proxy / LB with HSTS in production; not implemented locally (plain HTTP)
Encryption at rest	Documented decision — managed-Postgres volume encryption or SQLCipher in production; SQLite here is unencrypted
Password hashing; decay scheduling	Documented decision — PBKDF2 120k (argon2id in prod); explicit trigger, no cron
Token refresh / rotation / revocation	Known gap — no refresh flow, no denylist
Login rate limiting	Known gap
Redaction recall on unanticipated names	Known gap — lowercase/transliterated names in prose can survive
Scribe failure recovery	Known gap — synchronous; a crash mid-run loses the summary
Real speech recognition	Known gap — default recogniser is a simulated stub ; no audio has ever been transcribed by this build
Acoustic diarisation; consent artefact on patient recordings	Known gap — labels come from the transcript source; the clinician is a party and is never asked
Per-user normalisation of learning signals	Known gap — one enthusiast counts as consensus
Enum columns typed String not Enum	Known gap — structural fix for D-055
Formal accessibility / WCAG audit	Known gap — colour is never the sole signal, but no audit was run
Risk ordinal floored by deterministic rules	Implemented — a model may raise a level, never lower one; provenance stored per note (D-066)
Patient-facing generation	Implemented — structurally impossible, guarded at import, not gated by an approval step someone can click through (D-067)
Human-human contradiction detection	Implemented (narrow) — allergy/dose/status, deterministic, never auto-resolved (D-068)
Contradiction recall	Known gap — watchlist not formulary; absence of a flag is not evidence of agreement
Confidence calibration	Known gap — hedging density is a proxy, never validated against labelled data
Exposure bias in the learning loop	Partially mitigated — one reserved exploration slot; no off-policy evaluation (D-069)
Wire-format timezone correctness	Implemented (by convention) — UTC offsets via one annotation, pinned by tests that walk payloads; a new endpoint can still regress it (D-061)
End-to-end browser testing	Known gap — component tests mock Api, so a change to the fetch layer is caught by neither suite

Plainly: locally there is no TLS and no encryption at rest — plain HTTP on localhost, an unencrypted gitignored SQLite file. Both are deployment configuration rather than application code, hence decisions with a stated production shape; the honest consequence is that **this build is not safe for real PHI as-is**, which the README states too so it cannot be missed by someone who opens one file.

Verification. 435 backend tests plus 25 frontend component tests, no API key or network needed. Access-control and history tests were **deliberately broken to confirm they can fail** — reversing D-004 fails exactly the staff-visibility tests, removing the clinic filter fails 15, disabling the conflict guard fails 3. The same discipline applied to the Phase 7 regressions: 10 of 25 frontend tests and 10 of 15 backend ones fail when the code they cover is reverted. A security test that cannot fail is worse than none, because it is mistaken for coverage. Logs were grepped for planted names, identifiers and body text after exercising every route: zero hits.